SAFE LINK

Wearable-Based Group Safety and Participant Monitoring System

Project Proposal — Embedded Systems & IoT
August 2026

Contents

1. Executive Summary
2. Problem Statement
3. Target Users and Use Cases
4. Existing Challenge and Why Current Manual Monitoring Is Limited
5. Proposed Solution
6. Core Value Proposition
7. Detailed Feature Analysis
8. Organizer Dashboard Experience
9. Wearable Hardware Design
10. Sensor and Component Selection
11. Communication Architecture
12. Backend and Cloud Architecture
13. Database and Data Flow
14. Alert and Decision Logic
15. System Architecture Diagram
16. User Journey
17. Normal Operation Flow
18. Emergency Operation Flow
19. Prototype MVP Scope
20. Hardware Feasibility
21. Technical Risks and Limitations
22. False Alert and Failure Analysis
23. Privacy and Data Protection Considerations
24. Testing and Validation Plan
25. Future Development
26. Expected Impact
27. Conclusion

1. Executive Summary

SAFE LINK is a wearable-based safety and accountability system for organized outdoor groups — youth camps, university excursions, school field trips, hiking and scouting groups, outdoor training programs, and adventure tourism operators. Each participant wears a compact electronic device that reports status to a central system; organizers monitor the group through a digital dashboard rather than through continuous manual observation.

The system is built around a single design principle: **surface exceptions, not raw data** . Rather than asking an organizer to watch a stream of coordinates, heart rates, and battery levels, SAFE LINK converts sensor telemetry into a small set of operationally meaningful states — SAFE, WARNING, CRITICAL, and UNKNOWN — and only asks for attention when a participant's state changes for the worse. This allows a small organizing team to responsibly oversee a group far larger than they could track by headcount and radio alone.

The proposal covers eleven core features spanning identification, emergency signaling (SOS), location and last-known-location, movement and fall detection, heart-rate support monitoring, battery health, checkpoint attendance, connection status, geofencing, group-separation detection, and wearable-integrity (removal) detection. Each is analyzed against four questions: what problem it solves, what the organizer actually sees, how it is technically implemented, and what action the organizer can take as a result.

SAFE LINK is explicitly scoped as an **organized-group safety tool operating within a defined event area** — not a military system, not a medical diagnostic device, and not a general-purpose tracking product. The proposal distinguishes clearly between what a prototype (MVP) can realistically deliver using off-the-shelf embedded components (ESP32/STM32-class microcontrollers, a GNSS module, an IMU, an optical heart-rate sensor, and a low-power radio), what the fuller proposed architecture supports, and what remains future work. Every feature in this document is designed to fail safely: an unresponsive device is reported as unknown, never silently assumed to be fine.

2. Problem Statement

Organized outdoor activities routinely place a small number of adult supervisors in charge of dozens or hundreds of participants spread across open, visually obstructed, or physically dispersed terrain. The prevailing tools for keeping track of everyone — headcounts, buddy systems, handheld radios, and scheduled roll calls — are all manual, periodic, and fundamentally reactive: they surface a problem only when a person actively checks and notices something is wrong, often minutes after the underlying event occurred.

This creates several recurring failure modes. A participant who falls or becomes incapacitated on a trail may not be discovered until the next scheduled checkpoint. A child who wanders from their assigned group during free time may go unnoticed until an evening headcount comes up short, at which point the search has to start from the last confirmed sighting rather than a

recent position. A medical or safety event that a participant cannot verbally report — because they are alone, injured, or out of radio range — has no path to reach an organizer at all. Even when nothing goes wrong, organizers spend a disproportionate share of their attention on routine verification ("is everyone still here?") rather than on the exceptions that actually require judgment and intervention.

SAFE LINK is proposed as a response to this pattern: not a replacement for organizer judgment or physical supervision, but a system that continuously performs the low-value, high-frequency task of noticing — so that human attention is reserved for deciding what to do once something has been noticed.

3. Target Users and Use Cases

The system is designed to generalize across several categories of organized outdoor group, each of which shares the same underlying structure — one or a few responsible adults, many participants, an open or semi-open area — but differs in group size, participant age, activity duration, and terrain.

User Type	Representative Scenario	Primary Need
Youth camps	Week-long overnight camp, ages 8–14, multiple activity zones	Continuous headcount and fast SOS/ fall response for minors who may not self-report
University outdoor clubs	Weekend hiking or climbing trip, small self-organized groups	Separation detection on trail; reliable last-known-location if signal drops
School excursions	Day trip to a museum, park, or historical site with 100+ students	Rapid check-in and attendance verification at checkpoints
Scouting activities	Multi-day wilderness trip with staged campsites	Per-site geofencing; battery life across a multi-day itinerary
Field trips	Educational site visits with a fixed itinerary and volunteer chaperones	Simple, low-training-overhead operation
Outdoor training programs	Physically demanding courses — obstacle courses, endurance training	Fall and prolonged-inactivity detection; heart rate as a supporting signal
Adventure tourism operators	Paid guided excursions — rafting, zip-lining, guided trekking	SOS response and a defensible accountability/incident record

Despite their differences, every one of these contexts reduces to the same operational question that motivates the system's design: at any given moment, which participants require attention, and what is the fastest path from "something is wrong" to "an organizer knows and can act"?

4. Existing Challenge and Why Current Manual Monitoring Is Limited

Current practice across nearly all of the use cases above relies on some combination of headcounts, buddy systems, handheld radios, and verbal roll calls. Each of these is a

reasonable tool in isolation, but each has a structural limitation that SAFE LINK is designed to address without discarding the underlying practice — the system is intended to sit alongside these methods, not replace organizer judgment.

Method	What It Tells You	What It Misses
Headcount	Whether the expected number of people is present, at the moment counted	Who specifically is missing, where they were last seen, and how long they have been unaccounted for
Buddy system	Whether a peer has recently seen the participant	Depends on peer attentiveness and honesty; breaks down for younger participants or large groups
Handheld radio	Real-time voice contact with whoever is actively listening and near a radio	Does not scale past a few group leaders; requires someone to notice a problem and actively report it
Scheduled roll call	A confirmed, named list of who is present at that instant	Nothing between roll calls; time-consuming and disruptive if run frequently enough to close that gap

None of these methods are continuous, and none of them automatically surface *time since last confirmed status* — a quantity that matters enormously in an emergency but that no manual method tracks by default. SAFE LINK does not ask organizers to trust a machine over their own judgment; it removes the need to manually poll every participant in order to find out who, if anyone, currently needs that judgment applied.

5. Proposed Solution

SAFE LINK closes the loop between participant, wearable, backend system, and organizer dashboard. Each participant wears a compact device that collects safety-relevant signals — location, motion, connectivity, battery, and (as a supporting signal) heart rate — and transmits them to a central system. An alert engine continuously evaluates each participant's state against defined thresholds and raises an alert only when something falls outside the expected pattern. The organizer interacts with a dashboard, not with raw sensor feeds.

> *Enable organizers to quickly identify and respond to exceptions rather than manually monitoring every participant continuously.*

This exception-based philosophy governs every design decision in the sections that follow. A participant who is moving normally, within their geofence, with a charged battery and a healthy connection, produces no dashboard noise at all — their entry simply reads SAFE. The system is deliberately scoped as an organized-group safety and accountability tool operating within a defined event or activity area. It is not presented as a military-grade system, a medical diagnostic device, or a general-purpose global tracking product, and the technical implementation sections are written to avoid claiming capabilities in any of those directions.

6. Core Value Proposition

- **For organizers:** fewer manual headcounts and radio checks, materially faster awareness of emergencies, and a system that tells them who needs attention instead of requiring them to work it out.
- **For institutions and operators:** a documented, timestamped record of monitoring activity and incident response, supporting — though not by itself guaranteeing — a demonstrable duty-of-care process.
- **For families and stakeholders:** visible evidence that a structured, technology-assisted safety process exists alongside standard supervision practices.
- **For participants:** a low-friction, unobtrusive device that does not interfere with the activity itself — no app to operate, no continuous action required, only a button to press if something is genuinely wrong.

Underlying all four is the same structural shift: from continuously watching everyone, to being told when someone deviates from the expected pattern. This is what distinguishes SAFE LINK from "a collection of sensors connected to a website" — every hardware signal in the system exists because it feeds a specific, named organizer decision, not because it was easy to measure.

7. Detailed Feature Analysis

Each of the eleven core features below is analyzed against the same four questions: the problem it solves, what the organizer sees, how it is technically implemented, and what action the organizer can take as a result. Dashboard examples use a consistent illustrative participant (Participant 023, device WG-023, Hiking Team B) throughout for continuity.

Feature 1 — Participant Identification

Purpose

Every other feature in this system is only useful if an incoming signal can be traced to a specific person. Identification is the foundation the rest of the system is built on: it is what turns "a device reported a fall" into "Participant 023 may have fallen."

Organizer Dashboard

```
Participant: Participant 023
Device ID: WG-023
Group: Hiking Team B
Status: NORMAL
Last Update: 10:42:16
```

Technical Implementation

Each wearable carries a unique device identifier — either derived from the microcontroller's factory-programmed hardware ID (for example, the MAC-derived identifier available on ESP32-class chips) or a separately assigned serial number written to flash during manufacturing or setup. During pre-event registration, an organizer links a physical device to a participant record in the database, ideally by scanning a QR code printed on the wearable's strap with a

registration tablet or phone rather than typing IDs manually. Every telemetry and alert packet the device sends carries this device ID; the backend performs the participant lookup, meaning identity is a database-side association rather than something the device itself needs to know or compute.

This mechanism has a clear limitation, stated explicitly rather than glossed over: it proves which *device* transmitted the data, not that the correct person is physically wearing it. That gap is intentional — it is the reason Feature 11 (Wearable Integrity and Removal Detection) exists as a complementary control rather than being folded into identification itself.

Organizer Response

Organizers can reassign a device to a different participant record if devices are swapped, and the registration workflow flags any device transmitting without a matching participant assignment before the event begins.

Feature 2 — SOS Button

Purpose

Automatic detection cannot catch everything — a participant who feels unsafe, is lost but not yet flagged as separated, or is experiencing a non-fall medical issue needs a direct way to escalate. The SOS button is the participant's manual override on top of all the automatic monitoring described elsewhere in this document.

Organizer Dashboard

```
CRITICAL SOS ALERT

Participant: Participant 023
Device: WG-023
Group: Hiking Team B

Location:
[Current or Last Known Coordinates]

Time: 10:42:16

Status: ASSISTANCE REQUIRED
```

Technical Implementation

A dedicated physical button requires a deliberate long-press (on the order of one to two seconds) rather than a single tap, specifically to reduce accidental activation from a brushing branch or a shifted pack. On activation, the microcontroller immediately assembles a high-priority packet — bypassing its normal duty-cycle sleep schedule — tagged with an SOS flag, the current or most recent GPS fix, a timestamp, and battery status, and writes the event to local flash *before* attempting transmission, so the event survives even if the radio link fails at that instant. A short haptic pulse and LED flash confirm to the participant that the press registered, which matters because the device has no screen to otherwise show feedback.

The device retransmits at short intervals until it receives an application-layer acknowledgment

from the backend — not merely a radio-layer ACK from the gateway — because delivery to a gateway is not the same thing as an organizer having seen the alert. The device gives different feedback once acknowledgment is confirmed versus while still retrying. A short cancel window (releasing before the hold threshold, or a brief confirmation buzz with a follow-up cancel option) keeps a false trigger recoverable without requiring the participant to explain themselves after the fact.

Organizer Response

The dashboard visually prioritizes SOS above all other alert types and requires explicit organizer acknowledgment, recording who acknowledged it and when. The alert remains active on the dashboard — not auto-cleared — until the organizer logs a resolution.

Feature 3 — Location and Last Known Location

Purpose

Provide a participant's position when it is needed, and — just as importantly — preserve and clearly label the last successfully received position when live tracking is temporarily unavailable, so organizers are informed rather than left blind.

Organizer Dashboard

```
Current Location: 9.0357, 38.7651
Last Updated: 12 seconds ago
Location Status: CURRENT
```

```
LOCATION STATUS: LAST KNOWN LOCATION
Last Position: 9.0357, 38.7651
Last Received: 14 minutes ago
```

Technical Implementation

A GNSS receiver (a u-blox NEO-M8N–class module is a reasonable MVP choice) acquires a position fix at a configurable interval. Because continuous high-rate GPS acquisition is one of the largest single power draws in the wearable, the MVP uses an adaptive schedule rather than a fixed timer: fixes are taken more frequently — roughly every 15 to 30 seconds — while the device's own activity classifier (Feature 4) reports the MOVING state, and much less frequently — every several minutes — while STATIONARY, with the accelerometer's motion interrupt able to wake the GNSS early if movement resumes. Each fix is timestamped and attached to the regular telemetry packet. The backend independently stores the most recent *valid* fix per participant regardless of whether the latest packet contained a fresh one, so "last known location" persists automatically across any gap in reception.

Limitations are stated explicitly rather than assumed away: GNSS requires reasonable sky visibility, and dense forest canopy, canyons, and indoor structures will degrade or block a fix entirely; a cold-start acquisition can take significantly longer than a warm one in poor conditions. The dashboard is required to show a visible freshness indicator and must never silently reuse a stale point as though it were current — CURRENT and LAST KNOWN LOCATION are always distinct, clearly labeled states.

Organizer Response

When a participant shows LAST KNOWN rather than CURRENT, organizers treat the coordinate as a search starting point, not a live position, and can task nearby group members or staff to re-establish physical contact.

Feature 4 — Movement and Activity State Detection

Purpose

Raw accelerometer data is meaningless to an organizer glancing at a dashboard. This feature's job is to convert that raw signal into a small set of operational states an organizer can act on without any technical interpretation.

Organizer Dashboard

```
Activity Status: MOVING

WARNING: PROLONGED INACTIVITY
Duration: 18 minutes
Last Known Location: [Coordinates]
```

Technical Implementation

A 6-axis inertial measurement unit (an MPU6050-class accelerometer/gyroscope combination is a common, well-documented MVP choice) is sampled continuously by the microcontroller. Firmware computes rolling features over a short window — typically the magnitude and variance of acceleration over roughly thirty to sixty seconds — to classify MOVING versus STATIONARY, and raises PROLONGED INACTIVITY once the STATIONARY state persists beyond an activity-appropriate threshold. That threshold must be tunable per activity type: a short window suits an active hike, while a longer one is needed for a scheduled rest stop or overnight sleep period, where flagging every quiet moment would make the alert meaningless.

POSSIBLE FALL detection specifically requires a *combination* of signals: a short high-magnitude acceleration spike consistent with an impact, together with a rapid orientation change, followed by a subsequent stillness window. Requiring all three in sequence is what distinguishes a fall pattern from an ordinary jump, a fast sit-down, or a participant simply running — each of which can trigger one of the three signals in isolation. This is explicitly framed as a heuristic, not a confirmed diagnosis, which is why the system reports it as **Possible Fall Event** and always routes it to organizer verification rather than auto-escalating it as a confirmed injury.

Organizer Response

On a possible fall alert, the recommended first step is attempting radio or verbal contact through nearby group members before dispatching a response, since low-cost IMU-based heuristics are expected to produce a meaningful false-positive rate; if contact fails within a short window, the alert follows the same verify-then-escalate pattern as SOS.

Feature 5 — Heart Rate Monitoring

Purpose

Heart rate is included strictly as a supporting physiological signal — never as a medical diagnosis and never as proof that a participant is or is not alive. Framing this correctly is a safety requirement in its own right, not just a legal caveat.

Organizer Dashboard

```
Heart Rate: 78 BPM
Signal Quality: GOOD
Status: NORMAL RANGE
```

```
HEART RATE ALERT
Participant: Participant 023
Reading: Unavailable / Abnormal
Requires: Manual Verification
```

Technical Implementation

A reflective optical (PPG) sensor such as a MAX30102-class module reads pulse signal against wrist skin over I2C. To conserve power, the device takes a short measurement window every few minutes rather than sampling continuously, since continuous PPG capture with motion-artifact filtering is one of the more power- and processing-intensive subsystems in the wearable. Firmware and backend both apply basic signal-quality checks — using perfusion index or signal-amplitude thresholds — before publishing a reading, and report SIGNAL POOR or UNAVAILABLE rather than fabricating a number when skin contact or motion conditions are inadequate.

Wrist-worn optical PPG is known to degrade substantially under motion, which is precisely when a reading might matter most — this is a general limitation of the sensing approach, not specific to any one component choice, and the proposal does not claim otherwise. A missing or abnormal reading is routed to "manual verification required," never to an automated medical-emergency or fatality inference; this distinction protects against a serious failure mode in which the system would otherwise appear to claim something a $10 optical sensor and a loose strap cannot actually establish.

Organizer Response

Abnormal or missing heart-rate data is treated as one input among several — cross-referenced against recent movement, connection status, and whether an SOS has been raised — rather than acted on in isolation.

Feature 6 — Battery Monitoring

Purpose

A wearable with a dead battery looks, from the organizer's point of view, identical to a participant who is simply fine and has nothing to report — unless the system proactively flags the battery state before that happens.

Organizer Dashboard

```
Battery: 82%
Status: NORMAL
```

```
LOW BATTERY
Participant: Participant 023
Battery: 15%
Recommended Action: Recharge or replace device
```

Technical Implementation

State of charge is best measured with a dedicated fuel-gauge IC (a MAX17048-class coulomb counter is a common, low-cost MVP choice) rather than a bare voltage divider, because Li-ion cell voltage sags noticeably under load — particularly during a GPS or radio transmission current spike — which makes a simple voltage-to-percentage lookup noisy. If a fuel-gauge IC is not used, a voltage-divider approach is still workable but must be calibrated against a discharge curve specific to the chosen cell chemistry and capacity. Charging state is read directly from the charge-management IC's status output. Threshold alerts (for example, a 20% low-battery warning and a 5% critical warning) are computed from the reported percentage, either on-device or backend-side.

Organizer Response

A LOW BATTERY warning is routine triage — plan a swap or recharge at the next convenient stop. A CRITICAL battery warning is treated differently: it is a prompt to proactively check that participant's status *while there is still time to do so*, converting a battery warning into a location and status check rather than waiting for the device to go dark.

Feature 7 — Check-in / Attendance Status

Purpose

Verify that participants are accounted for at defined moments — an assembly, a meal, a return to camp — as an event-accountability tool, not a continuous surveillance feed and not a generic school-attendance system.

Organizer Dashboard

```
MORNING ASSEMBLY
Expected: 120
Checked In: 116
Missing: 4
```

```
Participant 023 — Last Known Location: Dormitory Area
Participant 041 — Device Offline
```

Technical Implementation

For the MVP, proximity-based BLE check-in is the recommended mechanism: a fixed BLE beacon or gateway is placed at the checkpoint, and wearables within range are automatically registered as present when an organizer opens a checkpoint session on the dashboard or a tablet, matched against the full expected roster so the system can name who is missing rather than only report a count. This is preferred over the alternatives for the MVP specifically because it requires no action from the participant — NFC or RFID check-in is more precise but requires

each participant to individually tap a reader, which becomes a bottleneck and a burden for younger children in a large group; QR-code organizer-scanned check-in is a reasonable manual fallback but adds staff time per participant. Proximity detection trades a small amount of boundary ambiguity — mitigated with a minimum dwell time or signal-strength threshold rather than a single ping — for check-in that happens automatically as the group gathers.

Organizer Response

The dashboard surfaces missing participants by name with their last known location, so follow-up begins immediately rather than requiring a manual cross-reference against a paper roster.

Feature 8 — Connection Status

Purpose

This feature exists to protect a single, critical distinction: **confirmed safe** is not the same thing as **we have not heard otherwise** . Treating silence as safety is a design error the system is built to avoid throughout.

Organizer Dashboard

```
Connection: ONLINE
Last Communication: 8 seconds ago
Signal: GOOD
```

```
STATUS: UNKNOWN — CONNECTION LOST
Last Contact: 10 minutes ago
Last Known Location: [Coordinates]
```

Technical Implementation

Lightweight heartbeat packets, sent independently of full telemetry, confirm a device is alive and in range even when it has nothing else to report. The backend tracks a last-successful-contact timestamp per device and applies a timeout — typically two to three times the expected heartbeat interval, to tolerate normal jitter and retries — before transitioning a participant through WEAK CONNECTION, DELAYED, and finally OFFLINE / STATUS UNKNOWN.

The governing design rule is stated plainly because it is easy to get wrong: **the absence of an alert must never be treated as equivalent to a confirmed-safe status.** The dashboard actively distinguishes a state that was recently confirmed from one that has simply gone quiet, rather than defaulting an unresponsive device to look identical to a healthy one.

Organizer Response

STATUS UNKNOWN triggers a manual-check task — radio the nearest group leader or dispatch someone — distinct from, and generally less urgent than, a CRITICAL alert, but never allowed to silently age out of view unresolved.

Feature 9 — Geofencing

Purpose

Convert "the area this activity is expected to stay within" into an automatic boundary check,

instead of relying on a group leader's visual judgment across an entire group's footprint.

Organizer Dashboard

```
GEOFENCE ALERT
Participant: Participant 023
Event: Exited Safe Zone
Time: 10:42:16
Location: [Coordinates]
```

Technical Implementation

Zones are defined as either a circle (center point plus radius — the simplest option, well suited to a single basecamp) or a polygon (an ordered list of coordinates — better suited to trail corridors or irregular property boundaries). The MVP recommendation is to evaluate zone membership **backend-side** against each incoming location fix rather than on-device: this centralizes the logic, allows zones to be edited mid-event without re-provisioning every wearable, and is simpler to implement and test on a compressed prototype timeline. On-device evaluation — which reduces alert latency and backend load at the cost of needing zone definitions synced to every wearable — is noted as a later optimization rather than an MVP requirement. The system should support multiple concurrent zones, including nested safe and restricted areas, with independently configurable alert severity.

Organizer Response

On exit, organizers see who left, from where, and how long ago, and can visually confirm on the map whether it reflects a genuine excursion or a GPS fix jittering right at a boundary edge before deciding how to respond.

Feature 10 — Grouping and Separation Detection

Purpose

Catch a participant drifting away from their assigned group before they become fully lost — where "lost" here is defined relative to the group's own position, not a fixed map boundary.

Organizer Dashboard

```
GROUP B
Members: 12
Within Expected Area: 11
Potentially Separated: 1
Participant: Participant 023
Distance from Group: 1.1 km
```

Technical Implementation

The specification explicitly rules out vague language such as "far away," so this feature requires a concrete, defined metric. The MVP recommendation is **distance from the group centroid** — the mean position of all currently-online members of the group — rather than distance from a designated leader (not every group structure names one, and a leader's device is a single point of failure for the whole metric) or distance from the nearest member (cheap to compute, but can mask an entire sub-group of two or three people drifting together, and is noisy for small

groups).

The centroid approach has its own known weakness, stated directly: a single member who is already far away pulls the centroid toward themselves, potentially masking their own separation. The mitigation is to compute the centroid from members within a normal spread of one another — for example, excluding outliers beyond roughly 1.5 times the median pairwise distance — before measuring anyone's distance from it. Thresholds must vary by activity type: a museum group might reasonably use 30–50 meters, while a hiking trail might allow several hundred meters of normal spacing along the trail itself. A distance threshold is always combined with a time threshold ("beyond X meters for more than Y minutes") specifically to avoid flagging the brief, normal spreading that happens when someone stops to tie a shoe or the trail bends.

Organizer Response

Organizers first check whether the flagged participant is together with any other named individual — a pair of stragglers reads very differently from a genuine solo separation — and radio the nearest group member before independently investigating.

Feature 11 — Wearable Integrity and Removal Detection

Purpose

Every feature above is only meaningful while the device is actually being worn. This feature closes that gap by detecting when the wearable comes off.

Organizer Dashboard

```
WEARABLE INTEGRITY ALERT
Participant: Participant 023
Event: DEVICE REMOVED OR TAMPERED
Time: 10:42:16
Last Location: [Coordinates]
Requires: Participant Verification
```

Technical Implementation

The recommended MVP mechanism is a closed conductive loop running through the strap's clasp: while fastened, the loop is closed and a GPIO pin reads continuity; opening the clasp breaks the loop and is detected immediately in firmware. This is favored for a student prototype over capacitive or optical skin-contact sensing — a fitness-tracker-grade approach that is more sensitive to strap tightness, skin tone, and moisture, and would require calibration effort better spent elsewhere on a compressed timeline. A magnetic reed-switch clasp and a mechanical tamper switch are viable alternatives with different trade-offs: a reed switch is very simple but only detects clasp state (not, for example, a cut strap), while a mechanical switch is robust but adds a moving part.

The specification requires distinguishing accidental detachment, legitimate removal (charging, bathing, a medical exam), and possible tampering — a distinction that hardware alone cannot fully resolve. The proposed approach combines the raw removal signal with context: whether it occurred during a scheduled charging window, at a designated rest area, or unexpectedly mid-

activity, to set alert severity, while keeping the alert an **accountability item requiring verification** , never an automatic emergency.

Organizer Response

Locate the participant, confirm status, and either have them re-wear the device — closing the loop clears the alert automatically — or log a supervised exception (for example, "removed for medical exam, participant confirmed present").

8. Organizer Dashboard Experience

The organizer dashboard is designed around decisions, not raw sensor values. Every element on screen exists to answer "who needs my attention, and what should I do about it" as directly as possible.

Summary Bar

The top of the dashboard always shows five counts: Total Participants, Safe/Normal, Warning, Critical, and Offline/Unknown. This is the single glance that tells an organizer whether anything currently needs attention before they look at anything else.

Participant List

Name	Group	Status	Battery	Connection	Last Update
Participant 023	Hiking Team B	NORMAL	82%	ONLINE	8s ago
Participant 041	Hiking Team B	UNKNOWN	61%	OFFLINE	10m ago
Participant 017	Cabin Group 3	WARNING	15%	ONLINE	22s ago

The list is sortable and filterable by status, so a busy organizer can immediately narrow to anyone not in the NORMAL state rather than scanning the full roster.

Interactive Map

The map shows live participant positions, defined safe and restricted zones, active alert locations, and last-known positions for anyone currently offline — all layered on a single view so spatial context (who is near whom, who is near a boundary) is immediately visible rather than requiring separate lookups.

Alert Center

A chronological, filterable feed of every alert: type, participant, severity, time, location, acknowledgment status, and a recommended next action. Critical alerts remain pinned at the top until acknowledged and resolved.

Participant Detail Page

Selecting any participant opens a full profile: identification, assigned group, current and last-

known location, activity state, heart rate, battery, connection status, wearable integrity status, recent alert history, and a historical event timeline — everything needed to make a decision about that one person without navigating away.

9. Wearable Hardware Design

A wrist-worn form factor is recommended over a chest strap or clip-on badge. Several features independently point the same direction: the optical heart-rate sensor (Feature 5) requires sustained skin contact best achieved at the wrist; the removal-detection loop (Feature 11) maps naturally onto a strap clasp; and a wristband is the most tolerant form factor for participants of varying ages and activity types. The enclosure should target a reasonable degree of dust and water resistance (an IP65-class rating is a sensible design goal) given outdoor use in variable weather, and should be low-profile enough not to interfere with normal activity. The SOS button should be large enough to operate without fine motor precision, while remaining recessed or requiring a deliberate long-press so it cannot be triggered by an accidental brush.

Wearable Internal Architecture

```
MICROCONTROLLER (ESP32-class)
  |-- GNSS module (UART) -- location fix
  |-- IMU: accelerometer + gyro (I2C) -- motion / fall detection
  |-- HR sensor: PPG (I2C) -- heart rate, supporting signal
  |-- Radio: BLE / LoRa -- wearable-to-gateway link
  |-- Battery + fuel gauge (I2C / ADC) -- power monitoring
  |-- I/O: SOS button, vibration motor, status LED, integrity-loop GPIO
```

The microcontroller is the single coordination point: it schedules sensor sampling, runs the on-device activity classifier, buffers events to local flash, manages the adaptive GNSS/HR sampling rates described in Sections 7 and 10, and drives the radio link. Everything else in this document — the alert logic, the dashboard, the communication architecture — is built on top of what this one chip reliably reports.

10. Sensor and Component Selection

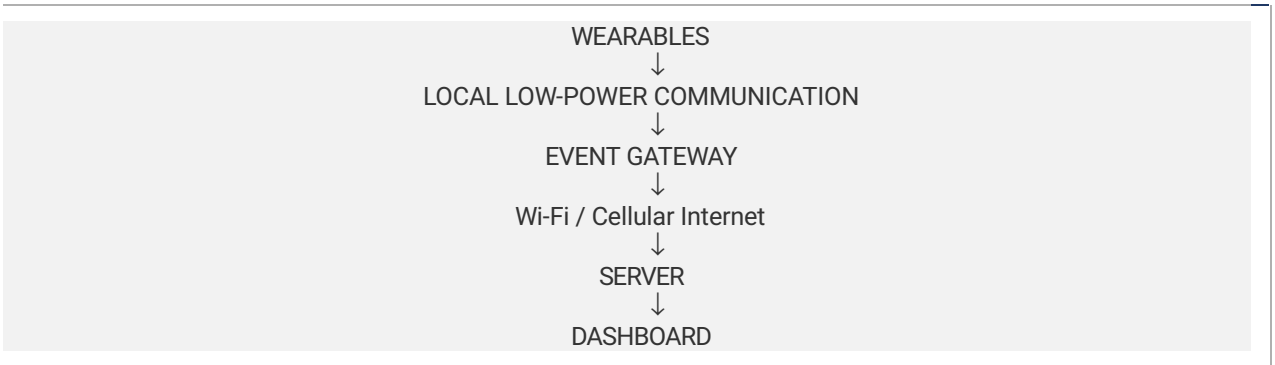
All components below are widely available, well-documented hobbyist-to-professional parts with mature software libraries — a deliberate choice to keep the prototype achievable on a student team's timeline rather than requiring custom silicon or exotic sourcing.

Component	Example Part	Interface	Role
Microcontroller	ESP32 (or STM32 + external radio)	—	Central processing, sensor scheduling, on-device classification, radio control. ESP32 is recommended for the MVP because its integrated Wi-Fi/BLE removes the need for a separate radio module.
GNSS receiver	u-blox NEO-6M / NEO-	UART	Position fix acquisition for Feature 3. NEO-

	M8N	(NMEA)	M8N supports multi-constellation reception for improved accuracy over NEO-6M.
IMU	MPU6050 (accelerometer + gyroscope)	I2C	Motion classification, fall-pattern detection (Feature 4).
Heart-rate / PPG sensor	MAX30102	I2C	Supporting physiological signal (Feature 5).
Battery + charge management	Single-cell Li-ion + TP4056-class charge IC	—	Power source and safe charging.
Fuel gauge	MAX17048-class coulomb counter	I2C	Accurate state-of-charge estimation (Feature 6), more reliable than raw voltage under load.
Integrity sensor	Conductive loop or reed switch in strap clasp	GPIO	Removal/tamper detection (Feature 11).
SOS input	Tactile momentary switch	GPIO (interrupt)	Manual emergency trigger (Feature 2).
Local feedback	Vibration motor + status LED	GPIO / PWM	Confirms SOS activation and device state to the participant without a screen.
Long-range radio (optional upgrade)	LoRa module, e.g. RFM95 / SX1276-class	SPI	Extends wearable-to-gateway range for spread-out trail deployments beyond BLE's effective reach.

For the MVP, ESP32's built-in BLE is recommended as the wearable-to-gateway link, with a dedicated LoRa module flagged as the deployment-specific upgrade path (Section 11) for use cases where participants may be spread across distances greater than BLE's typical open-air range.

11. Communication Architecture



The wearable communication layer connects each device to a nearby gateway using a short-range, low-power link. BLE is the recommended MVP default: it is simple, low-cost, and adequate for a compact, single-campsite footprint, with a typical open-air range on the order of

tens of meters. For deployments spread across a trail or a larger property where gateway placement is necessarily sparser, LoRa is the recommended upgrade path — a lower data-rate, longer-range link (realistically hundreds of meters to a few kilometers depending heavily on terrain and antenna placement, not a fixed guaranteed figure) better suited to that footprint. The choice should be made per deployment rather than assumed universally.

The gateway aggregates telemetry from every wearable in range and forwards it over a Wi-Fi or cellular backhaul to the cloud backend. Because backhaul availability cannot be assumed at every outdoor venue, the gateway buffers incoming data locally during any Internet outage and synchronizes once connectivity returns. This is deliberately a second layer of resilience on top of the wearable's own local event buffer (Section 7, SOS handling) — a temporary loss of the gateway's Internet connection does not mean data is lost, only delayed.

The proposal makes no claim of military-grade anti-jamming capability, satellite connectivity, or guaranteed operation in any environment; backhaul availability is deployment-specific and should be surveyed at each venue before an event, with the manual fallback procedures in Section 22 available as a matter of course rather than an exception.

12. Backend and Cloud Architecture

The backend consists of four logical components: an ingestion endpoint that receives packets from gateways, an alert engine that evaluates each participant's state against the thresholds described in Section 14, a data API that serves the dashboard (a REST interface for queries plus a push channel — WebSocket or equivalent — so the dashboard reflects new alerts immediately rather than polling), and an authentication layer restricting dashboard access to registered organizer accounts.

For a student prototype, a single small cloud instance with a managed database is sufficient: expected load is on the order of tens to a few hundred devices reporting on a multi-second to multi-minute cadence, not a scale that justifies a distributed microservice architecture or a managed message-queue cluster. Recommending that level of infrastructure for this stage would be poor engineering advice — it adds operational complexity without a corresponding benefit at MVP scale. A clear upgrade path (horizontal scaling of the ingestion and API layers, a managed queue between ingestion and the alert engine) exists if the system is later deployed across many simultaneous events, but it is explicitly out of scope for the prototype.

13. Database and Data Flow

Table	Key Fields	Purpose
Participants	participant_id, name, group_id	Roster and identity records
Devices	device_id, assigned_participant_id, last_seen	Device-to-participant mapping (Feature 1)
Groups	group_id, name, member_ids	Grouping for separation detection (Feature 10)

Geofences	zone_id, type (circle/polygon), coordinates, severity	Zone definitions (Feature 9)
Checkpoints	checkpoint_id, location, session_time	Check-in session records (Feature 7)
LatestState	participant_id, location, battery, hr, activity_state, connection_state	Current status per participant — what the dashboard reads directly
Alerts	alert_id, participant_id, type, severity, timestamp, acknowledged_by, resolution_notes	Full alert and response history
OrganizerUsers	user_id, name, role, access_level	Dashboard authentication and role-based access

Incoming telemetry updates the LatestState table directly — this is the record the dashboard reads, so the dashboard is always looking at current state rather than replaying a log. The alert engine evaluates each update against the rules in Section 14 and writes to the Alerts table only when a participant's state actually changes; a continuously-SAFE participant never produces an Alerts row. This is a deliberate data-minimization choice, consistent with the privacy principles in Section 23: the system stores current state and a record of exceptions, not a continuous raw location log.

14. Alert and Decision Logic

Severity	Triggers	Organizer Action Required
Critical	SOS activated; possible fall combined with prolonged inactivity; other high-confidence emergency combinations	Immediate acknowledgment and response
Warning	Geofence violation; low battery; prolonged inactivity; potential group separation; wearable removal	Verification and follow-up, not necessarily immediate dispatch
Information	Normal check-in; movement status changes; charging; group assignment	No action — visible in history, collapsed from the main alert view

This hierarchy exists specifically to prevent alert overload: without it, a dashboard reporting every state change with equal visual weight would train organizers to tune out alerts altogether, which is worse than not having the system at all. Representative decision rules, simplified for illustration:

```
IF impact_magnitude > IMPACT_THRESHOLD
  AND orientation_change > ORIENT_THRESHOLD
  AND subsequent_stillness_duration > STILL_WINDOW
THEN raise "POSSIBLE_FALL" (CRITICAL, requires verification)
```

```
IF distance(participant, group_reference) > SEPARATION_DISTANCE
  AND duration_exceeded > SEPARATION_TIME
THEN raise "POSSIBLE_SEPARATION" (WARNING)
```

```
IF time_since_last_contact > 3 * HEARTBEAT_INTERVAL
```

```
THEN set status -> "UNKNOWN" (WARNING — not SAFE)
```

Each participant is conceptually a small state machine (SAFE → WARNING → CRITICAL, with UNKNOWN as an orthogonal connectivity state). Anything reaching WARNING or above requires explicit organizer acknowledgment to clear — the system does not silently auto-resolve an alert once the underlying condition passes, since a state that resolved itself without anyone confirming why is exactly the kind of event that should be logged and reviewed, not erased.

15. System Architecture Diagram



Each layer maps directly to a section of this proposal: the sensor and microcontroller layers are detailed in Sections 9–10, local processing (on-device classification and buffering) is described throughout Section 7, the communication and gateway layers in Section 11, the backend, database, and alert engine in Sections 12–14, and the dashboard in Section 8. The chain is designed so that a failure at any single layer degrades gracefully rather than silently — a theme developed further in Section 22.

16. User Journey

1. **Pre-event registration** — devices are provisioned, paired to participant records, assigned to groups, and geofence zones are drawn for the event area.
2. **Event-day check-in** — an initial roster confirmation establishes that every registered device is online before the activity begins.
3. **Continuous monitoring** — throughout the activity, the system runs silently in the background, generating no dashboard noise for participants in the SAFE state.
4. **Exception handling** — as needed, following the alert flows described in Sections 17–18.
5. **Scheduled checkpoints** — attendance is verified at defined moments (Feature 7)

without interrupting the activity for a full manual roll call.

6. **Event close** — a final headcount, incident log export, and device recovery.
7. **Post-event** — data is retained for a defined review window and then deleted, per the privacy policy in Section 23.

17. Normal Operation Flow

This is the default path, and by design it is the path taken almost all of the time:

8. Wearable samples its sensors on the adaptive schedule described in Sections 7 and 10.
9. On-device processing classifies activity state and checks for any locally-detectable exception.
10. Telemetry is transmitted on its normal interval to the gateway.
11. The gateway relays the packet to the backend.
12. The alert engine evaluates the update and confirms the participant remains SAFE.
13. The dashboard's LatestState reflects the update; no alert is generated and no organizer action is required.

That the dashboard usually shows nothing happening is the intended, correct behavior of an exception-based system — it is a sign the system is working, not a sign it has nothing to do.

18. Emergency Operation Flow

Using an SOS/critical fall as the representative case:

14. Trigger — either a manual SOS press or an automatically detected possible fall.
15. The device buffers the event locally and assembles a high-priority packet, bypassing normal duty-cycle scheduling.
16. Prioritized transmission with retry-until-acknowledged, as described in Feature 2.
17. The backend flags the event CRITICAL immediately, bypassing normal batching.
18. The dashboard surfaces the alert prominently and requires explicit organizer acknowledgment before it can be cleared.
19. The organizer initiates a response following the guidance in the relevant feature section.
20. The resolution is logged with a timestamp and notes, closing the loop for later review and for the incident record described in Section 23.

19. Prototype MVP Scope

The specification requires a clear line between prototype capability and the fuller proposed

system. That line is drawn feature by feature below.

Feature	MVP Prototype	Future Enhancement
Identification	QR-assisted registration; device ID lookup	NFC/RFID-assisted registration
SOS	Long-press button, retry-until-ack	Multi-modal trigger options
Location	Adaptive-interval GNSS, last-known fallback	On-device path history, indoor positioning aids
Movement / fall detection	Threshold-based heuristic (impact + orientation + stillness)	ML-based classifier trained on pilot field data
Heart rate	Periodic PPG sampling with signal-quality gating	Improved motion-artifact filtering
Battery	Fuel-gauge based percentage and threshold alerts	Predictive remaining-runtime estimation
Check-in	BLE proximity with QR fallback	NFC/RFID checkpoint option
Connection status	Heartbeat + timeout classification	Signal-strength-based early warning before full loss
Geofencing	Circular zones, backend-side evaluation	Polygon zones, on-device evaluation
Separation detection	Centroid-based, outlier-excluded	Per-activity auto-tuned thresholds
Wearable integrity	Conductive loop in strap clasp	Context-aware severity (scheduled vs. unscheduled removal)
Communication	BLE to gateway, Wi-Fi/cellular backhaul	LoRa option for spread-out deployments

Every row in the "Future Enhancement" column is explicitly out of scope for the initial prototype and is not assumed or implied anywhere else in this document.

20. Hardware Feasibility

Every component in Section 10 is an off-the-shelf module with mature libraries and extensive hobbyist and professional documentation — a deliberate choice that keeps this achievable within a student team's timeline and skill set, requiring embedded firmware development, basic backend/web development, and basic hardware assembly rather than custom silicon design. Per-unit component cost at prototype quantities is difficult to state precisely given fluctuating supplier pricing, but is illustratively in the range of a few tens of US dollars per wearable, dominated by the GNSS and communication modules; this figure should be validated against current supplier quotes before committing to a build-out quantity rather than treated as fixed.

The recommended prototyping path is staged: individual subsystems proven on breadboards or off-the-shelf development boards first (GNSS fix acquisition, IMU threshold tuning, HR signal quality, battery discharge calibration, each independently), then integration onto a single custom PCB once the subsystem-level behavior is understood, and only then a fitted enclosure.

Building the full integrated board first, before any subsystem has been individually validated, is the most common way this kind of prototype loses time to debugging — the staged approach is recommended specifically to avoid that.

Fall detection accuracy, separation-detection thresholds, heart-rate signal quality under motion, and integrity-loop durability under repeated flexing are flagged here explicitly as the features most likely to need real field data before their thresholds can be trusted — a point developed further in Sections 21 and 24.

21. Technical Risks and Limitations

Risk	Impact	Mitigation
GNSS unavailable (canopy, indoors, canyons)	Location shows LAST KNOWN instead of CURRENT	Dashboard always labels freshness explicitly; never silently reuse a stale fix
Battery life vs. sampling rate	Higher-fidelity monitoring drains the battery faster	Adaptive sampling (Sections 7, 10) trades resolution for runtime automatically
False positive alerts (fall, separation)	Organizer attention consumed by non-events	Multi-signal fusion, tunable thresholds, "possible" framing requiring verification
Communication range / dead zones	Wearable drops to OFFLINE despite being fine	Local + gateway buffering, resync on reconnect, honest UNKNOWN state rather than assumed SAFE
Integrity-loop durability under repeated flexing	False removal alerts, or missed real removals over time	Mechanical design review during prototyping; flagged for field validation (Section 24)
HR signal quality during motion	Frequent UNAVAILABLE/ ABNORMAL readings while active	Signal-quality gating; treated as supporting signal only, never authoritative
Sensor/battery calibration drift over time and temperature	Gradually inaccurate battery percentage or fall thresholds	Periodic recalibration procedure; conservative default thresholds
Gateway as a single point of failure	All wearables behind a downed gateway appear OFFLINE simultaneously	Overlapping gateway coverage for larger events; dashboard should distinguish a mass-offline event from isolated ones

22. False Alert and Failure Analysis

The system's core defense against false alerts is structural, not just a matter of tuning: nothing is ever reported as a confirmed event, only as a "possible" one requiring verification, and multi-signal fusion (Section 7, Feature 4) is used specifically to keep single-sensor noise from reaching the dashboard as an alert at all. Thresholds are tunable per activity type rather than fixed globally, because a single global threshold is itself a common source of false positives across genuinely different contexts (a hike versus a rest period, for example).

Failure modes are treated as design cases, not edge cases dealt with after the fact. A device failure (dead battery, hardware fault) results in an OFFLINE / UNKNOWN status, never a silent default to SAFE — the same fail-safe principle that governs Feature 8 throughout. A gateway or network failure triggers local buffering and resync rather than data loss, as described in Section 11, with the dashboard reflecting a degraded-but-not-silent state during the outage. A backend failure is the one case not otherwise covered above: the dashboard should display a visible system-health banner ("data last updated Xs ago") whenever the backend itself stops updating, so organizers know to fall back to manual procedures rather than trusting a frozen screen that looks identical to a quiet, uneventful moment.

23. Privacy and Data Protection Considerations

All wearable-to-server transmissions are encrypted (AES-128 as a floor, with TLS on the gateway-to-backend hop as a second layer). The data model is deliberately minimal: the system stores current state and a record of exceptions (Section 13), not a continuous raw location log, which limits both the privacy exposure and the storage burden. Data is scoped to the event and auto-deleted after a defined retention window rather than retained indefinitely. Dashboard access is role-based and limited to registered organizer accounts — participants and the general public have no access to the monitoring data.

As a matter of good practice, families and guardians of minor participants should be informed in advance that a monitoring device will be worn during the event, consistent with the transparency this kind of system depends on for trust. Specific legal or regulatory obligations around children's data — which vary by jurisdiction and by institution — are outside the scope of this technical proposal and should be reviewed with the organizing institution's own counsel before deployment rather than assumed from this document.

24. Testing and Validation Plan

21. **Bench / unit testing** per subsystem — GNSS fix acquisition time, IMU threshold behavior, HR signal quality, battery discharge-curve calibration — tested independently before integration.
22. **Firmware integration testing** on a single assembled unit, verifying the subsystems behave correctly together.
23. **Controlled small-scale trial** — a handful of devices in a known, safe area, with staff deliberately simulating falls, removals, SOS presses, and separations to measure both detection rate and false-positive rate.
24. **Communication range and reliability testing at the actual intended venue** — range depends heavily on terrain and obstruction, so this cannot be validated in the lab alone.
25. **Organizer usability testing** of the dashboard with non-technical staff, since the system's value depends entirely on organizers being able to act on it quickly under pressure.

26. **A short supervised pilot** at an actual small-scale camp or trip, run with standard manual safety procedures still active in parallel — the system is not relied upon exclusively until it has demonstrated itself in that pilot.

Fall detection, separation thresholds, HR reliability under motion, and integrity-loop durability — the same items flagged in Section 20 — are the features most in need of this real-world validation before their default thresholds should be trusted beyond the pilot stage.

25. Future Development

- Solar-assisted charging for multi-day trips.
- ML-based fall and activity classification trained on pilot field data, to reduce false-positive rates below what fixed thresholds can achieve.
- NFC/RFID checkpoint option alongside BLE proximity check-in.
- Polygon geofencing with on-device evaluation.
- Satellite backhaul for fully remote deployments — flagged explicitly as requiring dedicated budget and hardware evaluation, not assumed feasible within the current scope.
- Integration hooks for existing camp-management or registration software.
- Multi-gateway redundancy for larger events.
- Institution-configurable data-retention policy settings.

26. Expected Impact

The most direct impact is response time: an exception that previously might not surface until the next scheduled headcount can instead reach an organizer within seconds of being detected. The second, less visible impact is on organizer workload — the gap between overseeing a group of fifteen and a group of a hundred and fifty is, under manual methods, largely a matter of how much continuous attention a person can sustain; an exception-based system compresses that gap by removing the need to manually verify participants who require no attention at all. A third impact is a better incident record: timestamped alerts and logged responses produce a more complete account of what happened and when than memory and verbal reports alone.

The underlying pattern — a few supervisors responsible for many people across a physical area — is not unique to outdoor camps, and the same architecture could plausibly extend to adjacent settings in the future. This proposal, however, is deliberately scoped to organized outdoor groups within a defined event area, and makes no claim beyond that scope.

27. Conclusion

SAFE LINK reframes participant safety monitoring from continuous manual vigilance to

exception-based response, built from feasible, well-documented embedded components and a clearly scoped MVP that separates what a prototype can deliver now from what remains future work. Every feature in this document connects a specific hardware signal to a specific organizer decision, from the SOS button through to wearable-integrity detection, and every claim about accuracy, range, or reliability is stated with its actual limitations rather than presented as guaranteed.

The recommended next step is to move from this proposal into the staged prototyping path described in Section 20: subsystem-level validation first, followed by integration, followed by the controlled and pilot testing described in Section 24 — building the evidence needed to trust the system's thresholds before it is relied upon in an actual event.